Trabalho Prático 1 - Geometria Computacional

Júlia Almeida Luna (2024014024)
julialuna@ufmg.br
Gabriel de Souza Gomes (2024005742)
gabriel2024@ufmg.br

1 Introdução

O presente relatório tem como objetivo apresentar a resolução do Trabalho Prático 1 da disciplina de Algoritmos 2, cujo tema central é a aplicação prática de algoritmos de geometria computacional por meio da construção de uma ferramenta interativa para consulta de bares participantes do Comida Di Buteco em Belo Horizonte.

O problema proposto consiste em receber um endereço informado pelo usuário, convertê-lo para coordenadas geográficas e, a partir dele, selecionar os bares localizados dentro de uma região de interesse. Na funcionalidade principal, essa região é um retângulo definido pelo usuário por meio de um valor numérico associado ao tamanho da busca. Os bares encontrados devem ser exibidos tanto no mapa, como pinos de localização, quanto em uma tabela contendo nome, endereço e distância em relação ao ponto informado.

A estrutura de dados escolhida para sustentar a busca espacial foi uma árvore k-dimensional, neste caso uma árvore bidimensional, pois cada bar pode ser representado por duas coordenadas: latitude e longitude. Além da busca ortogonal por retângulo, também foi implementada a atividade extra proposta no enunciado: a possibilidade de realizar a busca em uma região circular, isto é, selecionar todos os bares dentro de um raio determinado pelo usuário.

Neste relatório são descritos a modelagem adotada, a organização dos arquivos, os algoritmos implementados, a construção da interface, a adaptação feita para a busca circular e a análise de complexidade da solução proposta.

2 Modelagem

O sistema foi modelado a partir de três componentes principais: os dados geográficos dos bares, a estrutura de busca espacial e a interface interativa de visualização.

Cada bar é representado como um objeto da classe `Bar`, contendo quatro atributos: nome, endereço, latitude e longitude. Essa escolha separa as informações textuais, utilizadas principalmente na exibição da tabela, das informações numéricas, utilizadas nas buscas geométricas. Como a base original continha apenas os endereços, foi criada uma versão intermediária do arquivo de dados, chamada `butecos_com_coords.csv`, que acrescenta latitude e longitude a cada bar. Inicialmente, essas coordenadas foram obtidas por meio da API de geocodificação; nos casos em que a conversão automática não foi bem-sucedida, as coordenadas foram inseridas manualmente. Dessa forma, todos os 127 registros presentes no CSV passaram a possuir coordenadas válidas e puderam ser carregados na árvore.

Geometricamente, cada bar passa a ser visto como um ponto no plano bidimensional

$$p = (\text{latitude}, \text{longitude}).$$

Embora latitude e longitude não formem um plano euclidiano perfeito, essa representação é suficiente para organizar os pontos na árvore k-dimensional e para fazer podas espaciais durante a busca. Para o cálculo final das distâncias apresentadas ao usuário, foi utilizada uma fórmula específica para coordenadas geográficas, a fórmula de Haversine, discutida na seção da atividade extra.

A árvore implementada é uma árvore 2-dimensional. Em cada nível da recursão, alterna-se o eixo de comparação: nos níveis pares, a divisão é feita pela latitude; nos níveis ímpares, pela longitude. Os nós internos armazenam o valor da mediana no eixo correspondente, enquanto os nós folha armazenam efetivamente um objeto `Bar`. Assim, a árvore funciona como uma estrutura de particionamento recursivo do espaço: a cada nível, o conjunto de bares é dividido em duas regiões menores.

Na interface, o usuário informa um endereço, um valor numérico de busca e o formato da região desejada. Se o formato escolhido é retangular, o sistema calcula os limites mínimo e máximo de latitude e longitude e executa a busca ortogonal na árvore. Se o formato escolhido é circular, o sistema interpreta o valor digitado como raio em quilômetros e executa uma busca por região circular, retornando apenas os bares cuja distância real ao centro é menor ou igual ao raio.

3 Organização da Implementação

A implementação foi separada em arquivos com responsabilidades distintas, o que torna o projeto mais organizado e facilita a manutenção.

`bar.py`
: Define a classe `Bar`, usada para armazenar nome, endereço, latitude e longitude de cada estabelecimento.

`D2Tree.py`
: Contém a implementação da árvore bidimensional, da classe `Tree_node`, da classe `D2_tree`, da busca retangular, da busca circular e da função de cálculo de distância entre coordenadas geográficas.

`tree_instance.py`
: Lê o arquivo `butecos_com_coords.csv`, cria os objetos `Bar`, ignora registros sem latitude ou longitude e instancia a árvore usada pelo restante do sistema.

`mapAPI.py`
: Implementa a função responsável por converter um endereço textual em coordenadas geográficas usando a API do OpenStreetMap/Nominatim.

`main.py`
: Define a aplicação Dash, carrega o arquivo GeoJSON com os bairros de Belo Horizonte, monta o mapa com `dash-leaflet`, cria os campos de entrada e configura a tabela de resultados.

`callbacks.py`
: Contém a lógica de atualização dinâmica do mapa e da tabela. É nesse arquivo que o endereço informado pelo usuário é geocodificado, a árvore é consultada e os marcadores, o retângulo ou o círculo são redesenhados.

`conversions.py`
: Arquivo auxiliar usado para transformar dados geográficos dos bairros de Belo Horizonte em um arquivo GeoJSON. Embora não seja executado diretamente no fluxo principal do aplicativo, ele documenta o processo utilizado para gerar o arquivo `bairros_bh.geojson`.

`assets/style.css` e `assets/reset.css`
: Definem o estilo visual da página, incluindo cabeçalho, painel lateral, inputs, tabela, mapa e responsividade para telas menores.

Essa divisão deixa clara a separação entre dados, estrutura de dados, lógica de busca, interface e estilização.

4 Solução

4.1 Preparação dos Dados

A base original do trabalho contém o nome e o endereço dos bares, mas não contém diretamente suas coordenadas. Por isso, foi necessário realizar uma etapa prévia de conversão dos endereços para latitude e longitude. Essa conversão foi feita com apoio da API Nominatim, associada ao OpenStreetMap, que recebe uma string de endereço e retorna possíveis coordenadas correspondentes.

O arquivo `mapAPI.py` concentra essa funcionalidade na função `getCoordinates`. A função monta os parâmetros da requisição, envia o endereço informado pelo usuário, interpreta a resposta em formato JSON e retorna uma tupla `(lat, lon)` quando encontra um resultado válido. Também há tratamento de exceções para evitar que falhas externas da API interrompam a execução do programa.

Para o funcionamento cotidiano do sistema, os bares já são lidos a partir do arquivo `butecos_com_coords.csv`. Essa decisão evita que todos os endereços precisem ser convertidos novamente a cada execução do aplicativo. Assim, a API é usada principalmente para geocodificar o endereço digitado pelo usuário, enquanto os bares participantes já ficam armazenados com coordenadas prontas.

Além disso, o projeto inclui um arquivo GeoJSON com os bairros de Belo Horizonte. Esse arquivo é carregado em `main.py` e exibido como uma camada adicional no mapa, permitindo que o usuário visualize melhor o contexto geográfico da cidade.

4.2 Construção da Árvore Bidimensional

A árvore bidimensional é construída na classe `D2_tree`. O construtor recebe a lista de bares e chama o método recursivo `build_tree`. Em cada chamada, o algoritmo decide o eixo da divisão a partir da profundidade atual:

$$\text{axis} = \text{depth} \bmod 2.$$

Se `axis = 0`, os bares são ordenados pela latitude; se `axis = 1`, são ordenados pela longitude. Após a ordenação, o elemento mediano é usado para definir o valor armazenado no nó interno. Os elementos antes da mediana formam a subárvore esquerda e os elementos a partir da mediana formam a subárvore direita.

O caso base ocorre quando resta apenas um bar na lista. Nesse caso, o nó criado é uma folha e armazena diretamente o objeto `Bar`. Essa diferença entre nós internos e folhas é importante para a lógica de busca: os nós internos servem para podar regiões do espaço, enquanto as folhas representam candidatos reais a serem retornados.

Em termos conceituais, a construção da árvore divide sucessivamente o conjunto de bares em regiões alternadas por latitude e longitude. Como a mediana é utilizada em cada nível, a árvore tende a ficar balanceada, o que melhora o desempenho médio das consultas em comparação com uma varredura sequencial de todos os bares.

4.3 Busca Ortogonal Retangular

A busca retangular foi implementada no método `range_search_rectangule`. A entrada da função é formada pelo nó atual, pelos limites mínimo e máximo de latitude, pelos limites mínimo e máximo de longitude e pela profundidade atual da recursão. A região de interesse pode ser descrita como

$$R = [\mathrm{lat}_{min}, \mathrm{lat}_{max}] \times [\mathrm{lon}_{min}, \mathrm{lon}_{max}].$$

Quando a busca chega a uma folha, verifica-se se o bar armazenado está dentro dos limites do retângulo. Isso é feito por meio das desigualdades:

$$\mathrm{lat}_{min} \leq \mathrm{bar.latitude} \leq \mathrm{lat}_{max}$$
$$\mathrm{lon}_{min} \leq \mathrm{bar.longitude} \leq \mathrm{lon}_{max}.$$

Se ambas forem satisfeitas, o bar é retornado; caso contrário, a folha é descartada.

Nos nós internos, a busca usa o eixo correspondente à profundidade para decidir quais subárvores podem conter pontos dentro da região consultada. Se o eixo for latitude, compara-se o valor armazenado no nó com os limites `lat_min` e `lat_max`. A subárvore esquerda só precisa ser visitada se o limite inferior do retângulo puder alcançar aquela região; a subárvore direita só precisa ser visitada se o limite superior puder alcançá-la. A mesma ideia é aplicada para longitude nos níveis ímpares.

Desse modo, a busca evita visitar partes da árvore que estão completamente fora da região retangular. Essa é a principal vantagem da árvore k-dimensional no trabalho: em vez de testar todos os bares, a estrutura permite descartar grupos inteiros de pontos com base na geometria da consulta.

4.4 Interface e Atualização do Mapa

A interface foi construída com Dash e `dash-leaflet`. O arquivo `main.py` define o layout principal da aplicação, composto por um cabeçalho, uma área de mapa e um painel lateral. No mapa, são exibidas a camada base do Leaflet, a camada GeoJSON dos bairros de Belo Horizonte e uma camada dinâmica chamada `map-layer`, onde são inseridos os marcadores e a forma geométrica da consulta.

O painel lateral contém três elementos principais de entrada: um seletor entre retângulo e círculo, um campo para o endereço e um campo numérico para a diagonal ou raio. Abaixo desses campos, há uma tabela `DataTable` que recebe os resultados da busca.

A conexão entre a interface e a lógica de busca está no callback `update_map`, definido em `callbacks.py`. Sempre que o usuário altera o endereço, o valor da busca ou o formato da região, a função é chamada. Primeiro, o endereço é convertido em coordenadas. Depois, dependendo do formato selecionado, a árvore executa a busca retangular ou circular.

Os bares encontrados são ordenados pela distância em relação ao centro informado. Em seguida, são gerados os marcadores no mapa e os dados da tabela. Para cada bar exibido, a tabela contém o nome, o endereço e a distância calculada. Essa ordenação por distância torna a consulta mais útil para o usuário, pois os estabelecimentos mais próximos aparecem primeiro.

A função `update_rectangle` desenha o retângulo correspondente à busca ortogonal, enquanto a função `update_circle` desenha o círculo quando a atividade extra é selecionada. Assim, a resposta do sistema não é apenas textual, mas também visual, permitindo que o usuário veja diretamente a região consultada e os bares encontrados.

4.5 Atividade Extra: Busca Circular

Além da busca retangular obrigatória, foi implementada a busca circular, correspondente à atividade extra do trabalho. Para essa parte, seguimos a ideia indicada pelo livro *Advanced Algorithms and Data Structures*, de Marcello La Rocca, que apresenta a busca em regiões hiperesféricas na seção sobre *region search*. A diferença prática é que, no nosso caso, a região hiperesférica em duas dimensões corresponde a um círculo no mapa.

A implementação está no método `range_search_circle`. A entrada é composta pelo nó atual, pelo raio em quilômetros, pela latitude do centro, pela longitude do centro e pela profundidade da recursão. A estratégia combina duas ideias: primeiro, usa-se uma aproximação em graus de latitude e longitude para podar subárvores que não podem interceptar a região circular; depois, quando uma folha é alcançada, calcula-se a distância real entre o centro e o bar para decidir se ele pertence ou não ao círculo.

Como trabalhamos com latitude e longitude, não bastava aplicar diretamente a distância euclidiana usual. Por isso, ao pesquisar como determinar distâncias a partir de latitudes e longitudes, encontramos e adotamos a fórmula de Haversine. Essa fórmula calcula a distância aproximada entre dois pontos na superfície da Terra considerando sua curvatura. No código, usa-se o raio médio da Terra como

$$R = 6371 \text{ km}.$$

Dados dois pontos com latitudes φ_1, φ_2 e longitudes λ_1, λ_2, todos em radianos, a fórmula usada pode ser descrita como:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right),$$

$$d = 2R\arcsin(\sqrt{a}).$$

No método de busca circular, o algoritmo converte inicialmente o raio para deslocamentos aproximados de latitude e longitude:

$$\Delta lat \approx \frac{r}{111}$$

$$\Delta lon \approx \frac{r}{111\cos(\varphi_c)},$$

em que r é o raio em quilômetros e φ_c é a latitude do centro em radianos. Esses deslocamentos definem uma região de poda semelhante a uma caixa envolvente do círculo. A partir dela, a árvore decide se precisa visitar a subárvore esquerda, a subárvore direita ou ambas. Ao chegar a uma folha, a função `calculates_distance` calcula a distância pela fórmula de Haversine e só adiciona o bar ao resultado se

$$d(centro, bar) \leq r.$$

Na implementação, foi considerada ainda uma pequena tolerância de 0,05 km, isto é, 50 metros, para evitar que erros de arredondamento ou pequenas imprecisões na conversão geográfica excluam bares que estejam praticamente sobre a fronteira do círculo.

Dessa forma, a atividade extra mantém a estrutura geral ensinada para buscas em regiões hiperesféricas, mas adapta o critério final de pertencimento ao contexto real de coordenadas geográficas.

5 Análise de Complexidade

5.1 Carregamento dos Dados

A leitura do arquivo CSV em `tree_instance.py` percorre todos os registros uma única vez. Para cada linha, o algoritmo verifica se existem latitude e longitude e, em caso positivo, cria um objeto `Bar`. Sendo n o número de bares no arquivo, o custo dessa etapa é

$$O(n).$$

O espaço também é $O(n)$, pois os bares válidos são armazenados em uma lista antes da construção da árvore.

5.2 Construção da Árvore

A construção da árvore k-d é realizada pela função `build_tree`. A lista de bares é ordenada duas vezes: uma por latitude e outra por longitude, com custo $O(n \log n)$.

Em cada chamada recursiva, a função recebe duas listas do mesmo subconjunto: uma ordenada pelo eixo atual e outra pelo eixo alternativo. A mediana do eixo atual é usada para criar o nó interno e dividir os bares entre as subárvores esquerda e direita. Em seguida, a lista do eixo alternativo é particionada linearmente, preservando a ordenação necessária para o próximo nível.

Assim, após as ordenações iniciais, cada nível da recursão realiza trabalho total linear. A recorrência é

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Pelo Teorema Mestre,

$$T(n) = O(n \log n).$$

Portanto, a construção da árvore tem complexidade $O(n \log n)$. Em memória, a árvore armazena $O(n)$ nós e utiliza listas auxiliares nas subdivisões, resultando em espaço adicional $O(n)$, além da pilha recursiva de profundidade $O(\log n)$.

5.3 Busca Retangular

A busca retangular visita apenas os ramos da árvore cujas regiões ainda podem intersectar o retângulo consultado. Em uma árvore balanceada de duas dimensões, o comportamento esperado da busca ortogonal é significativamente melhor do que testar todos os pontos. Para uma k-d tree bidimensional, o custo típico de uma busca retangular pode ser descrito como

$$O(\sqrt{n} + k),$$

em que k é o número de pontos retornados. O termo $\sqrt{n}$ corresponde aos nós visitados na fronteira da região, enquanto k corresponde aos bares efetivamente encontrados.

A memória adicional da busca é proporcional à profundidade da recursão e ao tamanho da lista de resultados. Como a árvore tende a ser balanceada, a profundidade é $O(\log n)$, e a lista de saída ocupa $O(k)$. Assim, a memória adicional é $O(\log n + k)$.

5.4 Busca Circular

A busca circular utiliza a mesma árvore, mas altera a condição de poda e o teste final. Primeiro, o raio é convertido para deslocamentos aproximados de latitude e longitude, criando uma caixa envolvente da região circular. Essa caixa é usada para decidir quais ramos da árvore precisam ser visitados. Ao chegar às folhas candidatas, a fórmula de Haversine é aplicada para verificar se o bar realmente está dentro do raio.

Se m é o número de folhas candidatas visitadas após as podas, o custo do cálculo exato das distâncias é $O(m)$, pois a fórmula de Haversine tem custo constante por bar. Assim, o comportamento esperado é semelhante ao da busca retangular, acrescido do teste de distância nos candidatos:

$$O(\sqrt{n} + m),$$

com $m \geq k$, pois alguns bares dentro da caixa envolvente podem estar fora do círculo e serem descartados pelo teste final.

A memória adicional também é $O(\log n + k)$, considerando a pilha da recursão e a lista de bares retornados.

5.5 Atualização da Interface

Após a busca na árvore, o callback ordena os bares encontrados por distância até o centro informado. Se k bares são retornados pela busca, a ordenação custa

$$O(k \log k).$$

A criação dos marcadores e a montagem dos dados da tabela percorrem a lista de resultados uma vez, com custo $O(k)$. Portanto, após a busca espacial, o custo dominante da atualização visual é a ordenação por distância.

A geocodificação do endereço informado pelo usuário depende de uma API externa e, portanto, não é analisada como parte da complexidade algorítmica interna do programa.

6 Considerações Finais

A realização deste trabalho prático foi uma oportunidade importante para compreender como estruturas de dados geométricas podem ser aplicadas em sistemas interativos e próximos de um contexto real. A implementação da k-d tree tornou mais clara a ideia de particionamento recursivo do espaço e mostrou como a alternância entre latitude e longitude permite reduzir o número de pontos examinados em uma consulta.

O principal desafio esteve em conectar a parte teórica da disciplina com dados geográficos reais. Diferentemente de exercícios puramente cartesianos, o problema exigiu lidar com endereços, APIs de geocodificação, coordenadas de latitude e longitude, arquivos GeoJSON, visualização em mapa e atualização dinâmica da interface. Isso tornou a solução mais completa, mas também exigiu decisões adicionais de modelagem.

A busca retangular implementada atende à proposta principal do trabalho ao localizar bares dentro de uma região definida pelo usuário e exibi-los no mapa e na tabela. A atividade extra com círculo ampliou a solução, pois exigiu adaptar a ideia de busca em região hiperesférica à realidade de coordenadas geográficas. A adoção da fórmula de Haversine foi essencial nessa etapa, pois permitiu calcular distâncias mais adequadas entre pontos reais da superfície terrestre.

De forma geral, o projeto consolidou conhecimentos sobre árvores k-dimensionais, busca ortogonal, poda espacial e análise de complexidade, além de reforçar a importância de integrar algoritmos eficientes com interfaces compreensíveis para o usuário final.

7 Referências

Marcello La Rocca. *Advanced Algorithms and Data Structures.* Manning Publications, 2021.

OpenStreetMap. *Nominatim: Search API.* Disponível em: `https://nominatim.org/release-docs/latest/api/Search/`.

Dash. *Dash Documentation.* Disponível em: `https://dash.plotly.com/`.

Dash Leaflet. *dash-leaflet Documentation.* Disponível em: `https://www.dash-leaflet.com/`.

Leaflet. *Leaflet Documentation.* Disponível em: `https://leafletjs.com/`.